

บทที่ 10

คอลเล็กชัน (Collection)

วัตถุประสงค์

เพื่อให้ผู้เรียน

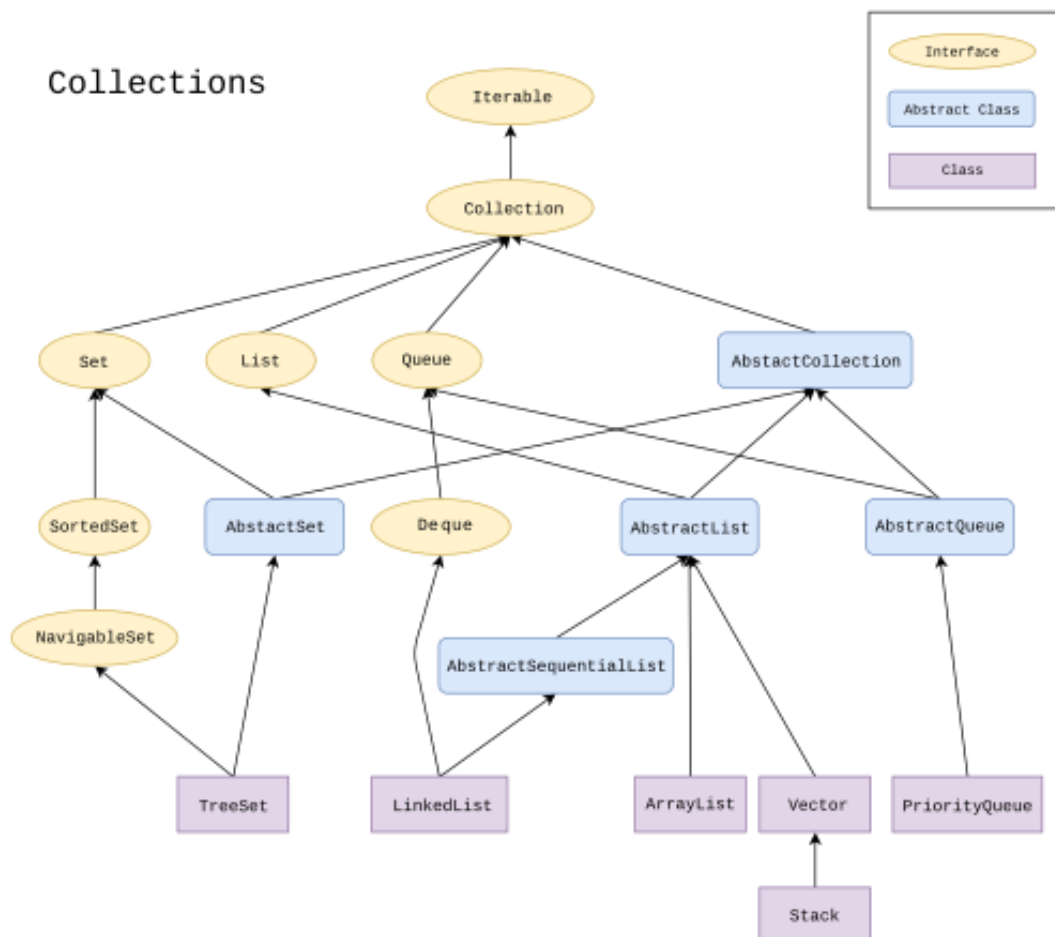
1. มีความรู้และอธิบายเกี่ยวกับความหมายความจำเป็นของโครงสร้างข้อมูลชนิดข้อมูลนามธรรมคอลเล็กชันอย่างถูกต้อง
2. มีความรู้เบื้องต้น อธิบายและยกตัวอย่าง และเขียนโปรแกรมโดยใช้โครงสร้างข้อมูลคอลเล็กชันได้
3. มีความรู้เบื้องต้น อธิบายและยกตัวอย่าง และเขียนโปรแกรมโดยใช้โครงสร้างข้อมูลสแต็คคอลเล็กชันได้
4. สามารถอธิบาย ยกตัวอย่างการประยุกต์ใช้โครงสร้างข้อมูลคอลเล็กชันด้วยการเขียนโปรแกรมได้

บทนี้นำเสนอภาพรวมของชนิดข้อมูลนามธรรม Collection และคลาส Collections ซึ่งนำเสนอการเชื่อมโยงกันในรูปแบบของอินเทอร์เฟซ และคลาสต่างๆ ในโครงสร้างข้อมูลชนิดนามธรรมที่ได้นำเสนอไป เนื้อในบทนี้ยังไปสู่การจัดเรียงและการค้นหา โดยใช้โครงสร้างดังกล่าวนี้ด้วย

เนื้อเริ่มต้นนำเสนอจาก ความรู้เบื้องต้นเกี่ยวกับคอลเล็กชัน ภาพความเชื่อมโยงของอินเทอร์เฟซและคลาสโครงสร้างข้อมูลชนิดนามธรรมที่มีในภาษาจาวา แสดงตัวอย่างการใช้งานคอลเล็กชัน ส่วนท้ายนำเสนอการใช้งานสแต็คคลาสที่สนับสนุนการจัดเรียงและการค้นหาข้อมูล

10.1 ความรู้เบื้องต้นเกี่ยวกับคอลเลกชัน

คอลเลกชัน (Collection) คือ อินเทอร์เฟซที่ให้โครงสร้างชนิดนามธรรมอื่นๆ ที่เป็นแบบข้อมูลแบบชุดได้นำไปอิมพลีเมนต์ใช้งาน ซึ่งพบว่าโครงสร้างข้อมูลที่น่าเสนอผ่านมาโดยส่วนมากนั้นเป็นโครงสร้างที่อิมพลีเมนต์อินเทอร์เฟซนี้ใช้งาน จะอยู่ไลบรารีของภาษา `java.util.Collection` แสดงแนวคิดดังรูปต่อไปนี้



รูปที่ 10.1 แสดงการเชื่อมโยงของโครงสร้างมูลชนิดคอลเลกชัน
(ที่มา : https://en.wikipedia.org/wiki/File:Java.util.Collection_hierarchy.svg)

จากภาพแสดงให้เห็นว่าคลาส `TreeSet` `LinkedList` `ArrayList` `Stack` `Vector` `PriorityQueue` ต่างมีส่วนการอิมพลีเมนต์ส่วนของคอลเลกชันทั้งสิ้น ดังนั้น การใช้งานคอลเลกชันต่างใช้งานผ่านโครงสร้างพื้นฐานดังกล่าวนี้ การเรียกใช้งานจะ `import`

java.util.Collection และประกาศอ็อบเจกต์ด้วยโครงสร้างข้อมูลที่เป็นคลาสต่างๆ เพื่อเก็บข้อมูล เมื่อระบุชนิดเป็นคอลเลกชัน จะมีเมธอดใช้งานดังนี้

● add(Object e)	boolean		
● addAll(Collection c)	boolean		
● clear()	void		
● contains(Object o)	boolean		
● containsAll(Collection c)	boolean	● parallelStream()	Stream
● equals(Object o)	boolean	● remove(Object o)	boolean
● forEach(Consumer action)	void	● removeAll(Collection c)	boolean
● getClass()	Class<?>	● removeIf(Predicate filter)	boolean
● hashCode()	int	● retainAll(Collection c)	boolean
● isEmpty()	boolean	● size()	int
● iterator()	Iterator	● splitterator()	Splitterator
● notify()	void	● stream()	Stream
● notifyAll()	void	● toArray()	Object[]
● parallelStream()	Stream	● toArray(IntFunction generator...Object[])	
● remove(Object o)	boolean	● toArray(Object[] a)	Object[]
● removeAll(Collection c)	boolean		
● removeIf(Predicate filter)	boolean		

รูปที่ 10.2 แสดงเมธอดอ็อบเจกต์คอลเลกชัน

ตัวอย่างที่ 10.1 การประกาศใช้งาน Collection และการเข้าถึง

```

1. package lesson10collections;
2. import java.util.ArrayList;
3. import java.util.Collection;
4. import java.util.Collections;
5. import java.util.LinkedList;
6. import java.util.List;
7. import java.util.PriorityQueue;
8. import java.util.Stack;
9. import java.util.TreeSet;
10. import java.util.Vector;
11. public class CollectionDemo {
12.     public static void main(String args[]){
13.         Collection cAL = new ArrayList<Integer>();

```

```

14. Collection cTS = new TreeSet<Integer>();
15. Collection cLL = new LinkedList<Integer>();
16. Collection cV = new Vector<Integer>();
17. Collection cST = new Stack<Integer>();
18. Collection cPQ = new PriorityQueue<Integer>();
19. cAL.add(50); cAL.add(60); cAL.add(5); cAL.add(2); cAL.add(1);
20. cTS.add(50); cTS.add(60); cTS.add(5); cTS.add(2); cTS.add(1);
21. cLL.add(50); cLL.add(60); cLL.add(5); cLL.add(2); cLL.add(1);
22. cV.add(50); cV.add(60); cV.add(5); cV.add(2); cV.add(1);
23. cST.add(50); cST.add(60); cST.add(5); cST.add(2); cST.add(1);
24. cPQ.add(50); cPQ.add(60); cPQ.add(5); cPQ.add(2); cPQ.add(1);
25. System.out.println(cAL);
26. System.out.println(cTS);
27. System.out.println(cLL);
28. System.out.println(cV);
29. System.out.println(cST);
30. System.out.println(cPQ);
31. }
32. }

```

ผลการรันโปรแกรม

[50, 60, 5, 2, 1]

[1, 2, 5, 50, 60]

[50, 60, 5, 2, 1]

[50, 60, 5, 2, 1]

[50, 60, 5, 2, 1]

[1, 2, 50, 60, 5]

ตัวอย่างที่ 10.2 แสดงการจัดการแบบกลุ่มข้อมูล

```
1. package lesson10collections;
2. import java.util.ArrayList;
3. import java.util.Collection;
4. import java.util.Collections;
5. import java.util.LinkedList;
6. import java.util.List;
7. import java.util.PriorityQueue;
8. import java.util.Stack;
9. import java.util.TreeSet;
10. import java.util.Vector;
11. public class CollectionDemo1 {
12. public static void main(String args[]){
13. Collection cAL = new ArrayList<Integer>();
14. Collection cTS = new TreeSet<Integer>();
15. Collection cLL = new LinkedList<Integer>();
16. Collection cV = new Vector<Integer>();
17. Collection cST = new Stack<Integer>();
18. Collection cPQ = new PriorityQueue<Integer>();
19. cAL.add(1); cAL.add(2); cAL.add(3); cAL.add(4); cAL.add(5);
20. cTS.add(6); cTS.add(7); cTS.add(8); cTS.add(9); cTS.add(10);
21. cLL.add(11); cLL.add(12); cLL.add(13); cLL.add(14); cLL.add(15);
22. cV.add(16); cV.add(17); cV.add(18); cV.add(19); cV.add(20);
23. cST.add(21); cST.add(22); cST.add(23); cST.add(24); cST.add(25);
24. cPQ.add(26); cPQ.add(27); cPQ.add(28); cPQ.add(29); cPQ.add(30);
25. System.out.println(" cAL:"+cAL);
26. System.out.println(" cTS:"+cTS);
27. System.out.println(" cLL:"+cLL);
```

```

28. System.out.println(" cV:" + cV);
29. System.out.println(" cST:" + cST);
30. System.out.println(" cPQ:" + cPQ);
31. Collection combine = new LinkedList<Integer>();
32. combine.addAll(cAL); combine.addAll(cTS); combine.addAll(cLL);
    combine.addAll(cV); combine.addAll(cST); combine.addAll(cPQ);
33. System.out.println(combine);
34. System.out.println(combine.containsAll(cV));
35. combine.removeAll(cV);
36. System.out.println(combine);
37. }
38. }

```

ผลการรันโปรแกรม

cAL:[1, 2, 3, 4, 5]

cTS:[6, 7, 8, 9, 10]

cLL:[11, 12, 13, 14, 15]

cV:[16, 17, 18, 19, 20]

cST:[21, 22, 23, 24, 25]

cPQ:[26, 27, 28, 29, 30]

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30]

true

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30]

10.2 การใช้งานคลาสสแตติกคอลเลกชัน

คลาสสแตติกคอลเลกชัน (Collections) มาจาก java.util.Collections ซึ่งมีให้ใช้งานได้โดยไม่ต้องประกาศตัวแปรอ็อบเจกต์ใหม่ แต่จะเป็นคลาสแบบสแตติก กล่าวคือ คลาสนี้มีให้

ใช้งานเพื่อจัดการข้อมูลชนิดที่เก็บข้อมูลแบบคอลเลกชันให้ได้สะดวกขึ้น คลาสดังกล่าวนี้มีเมธอดดังรูปต่อไปนี้

 EMPTY_LIST	List
 EMPTY_MAP	Map
 EMPTY_SET	Set
 addAll (Collection<? super T> c, T... elements)	boolean
 asLifoQueue (Deque<T> deque)	Queue<T>
 binarySearch (List<? extends Comparable<? super T>> list, T key)	int
 binarySearch (List<? extends T> list, T key, Comparator<? super T> c)	int
 checkedCollection (Collection<E> c, Class<E> type)	Collection<E>
 checkedList (List<E> list, Class<E> type)	List<E>
 checkedMap (Map<K, V> m, Class<K> keyType, Class<V> valueType)	Map<K, V>
 checkedNavigableMap (NavigableMap<K, V> m, Class<K> keyType, Class<V> valueType)	NavigableMap<K, V>
 checkedNavigableSet (NavigableSet<E> s, Class<E> type)	NavigableSet<E>
 checkedQueue (Queue<E> queue, Class<E> type)	Queue<E>
 checkedSet (Set<E> s, Class<E> type)	Set<E>
 checkedSortedMap (SortedMap<K, V> m, Class<K> keyType, Class<V> valueType)	SortedMap<K, V>
 checkedSortedSet (SortedSet<E> s, Class<E> type)	SortedSet<E>
 copy (List<? super T> dest, List<? extends T> src)	void

รูปที่ 10.3 (a) แสดงเมธอดของคลาสสแตติกคอลเลกชัน

 disjoint (Collection<?> c1, Collection<?> c2)	boolean
 emptyEnumeration ()	Enumeration<T>
 emptyIterator ()	Iterator<T>
 emptyList ()	List<T>
 emptyListIterator ()	ListIterator<T>
 emptyMap ()	Map<K, V>
 emptyNavigableMap ()	NavigableMap<K, V>
 emptyNavigableSet ()	NavigableSet<E>
 emptySet ()	Set<T>
 emptySortedMap ()	SortedMap<K, V>
 emptySortedSet ()	SortedSet<E>
 enumeration (Collection<T> c)	Enumeration<T>
 fill (List<? super T> list, T obj)	void
 frequency (Collection<?> c, Object o)	int
 indexOfSubList (List<?> source, List<?> target)	int
 lastIndexOfSubList (List<?> source, List<?> target)	int
 list (Enumeration<T> e)	ArrayList<T>

รูปที่ 10.3 (b) แสดงเมธอดของคลาสสแตติกคอลเลกชัน

max (Collection<? extends T> coll)	T
max (Collection<? extends T> coll, Comparator<? super T> comp)	T
min (Collection<? extends T> coll)	T
min (Collection<? extends T> coll, Comparator<? super T> comp)	T
nCopies (int n, T o)	List<T>
newSetFromMap (Map<E, Boolean> map)	Set<E>
replaceAll (List<T> list, T oldVal, T newVal)	boolean
reverse (List<?> list)	void
reverseOrder ()	Comparator<T>
reverseOrder (Comparator<T> cmp)	Comparator<T>
rotate (List<?> list, int distance)	void
shuffle (List<?> list)	void
shuffle (List<?> list, Random rnd)	void
singleton (T o)	Set<T>
singletonList (T o)	List<T>
singletonMap (K key, V value)	Map<K, V>
sort (List<T> list)	void

รูปที่ 10.3 (c) แสดงเมธอดของคลาสสแตติกคอลเลกชัน

sort (List<T> list, Comparator<? super T> c)	void
swap (List<?> list, int i, int j)	void
synchronizedCollection (Collection<T> c)	Collection<T>
synchronizedList (List<T> list)	List<T>
synchronizedMap (Map<K, V> m)	Map<K, V>
synchronizedNavigableMap (NavigableMap<K, V> m)	NavigableMap<K, V>
synchronizedNavigableSet (NavigableSet<T> s)	NavigableSet<T>
synchronizedSet (Set<T> s)	Set<T>
synchronizedSortedMap (SortedMap<K, V> m)	SortedMap<K, V>
synchronizedSortedSet (SortedSet<T> s)	SortedSet<T>
unmodifiableCollection (Collection<? extends T> c)	Collection<T>
unmodifiableList (List<? extends T> list)	List<T>
unmodifiableMap (Map<? extends K, ? extends V> m)	Map<K, V>
unmodifiableNavigableMap (NavigableMap<K, ? extends V> m)	NavigableMap<K, V>
unmodifiableNavigableSet (NavigableSet<T> s)	NavigableSet<T>
unmodifiableSet (Set<? extends T> s)	Set<T>
unmodifiableSortedMap (SortedMap<K, ? extends V> m)	SortedMap<K, V>

รูปที่ 10.3 (d) แสดงเมธอดของคลาสสแตติกคอลเลกชัน

ยกตัวอย่างการการจัดเรียงข้อมูลด้วยการเรียกใช้จาก java.util.Collections ให้เรียกใช้งาน ซึ่งสามารถเรียกใช้ด้วยเมธอด `.sort` ให้จัดเรียงข้อมูลอัตโนมัติ แต่ต้องจัดเตรียมข้อมูลให้เป็นรายการต่อเนื่อง java.util.List, java.util.ArrayList จึงสามารถเรียกใช้เมธอด `.sort` ของ java.util.Collections ได้ ต่อไปนี้จะแสดงตัวอย่างการใช้งานเมธอดที่กล่าวไว้พอสังเขป

ตัวอย่างที่ 10.3 แสดงการจัดเรียงข้อมูล 31, 11, 71, 10, 16, 91, 77, 12, 55, 50

```
1. package lesson10collections;
2. import java.util.Collections;
3. import java.util.List;
4. import java.util.ArrayList;
5. public class javaSortADT {
6.     public static void main(String args[]){
7.         int DATA[]={31, 11, 71, 10, 16, 91, 77, 12, 55, 50};
8.         List<Integer> a = new ArrayList<Integer>();
9.         System.out.println("ก่อนการจัดเรียง");
10.        for(int i=0;i<DATA.length;i++)
11.            System.out.print(DATA[i]+" ");
12.        //คัดลอกข้อมูล DATA ไปยัง List
13.        for(int i=0;i<DATA.length;i++){
14.            a.add(DATA[i]);
15.        }
16.        Collections.sort(a); //จัดเรียงข้อมูลใน List
17.        // คัดลอกข้อมูลกลับคืน DATA
18.        for(int j=0;j<a.size();j++){
19.            DATA[j]=a.get(j);
20.        }
21.        System.out.println("\nหลังการจัดเรียง");
22.        for(int i=0;i<DATA.length;i++)
23.            System.out.print(DATA[i]+" ");
24.        }
25.    }
```

ผลการรันโปรแกรม

ก่อนการจัดเรียง

31 11 71 10 16 91 77 12 55 50

หลังการจัดเรียง

10 11 12 16 31 50 55 71 77 91

อธิบายเพิ่มเติมสำหรับโปรแกรมดังกล่าวนี้ พบว่า มีขั้นตอนสำหรับการจัดเรียงข้อมูลรายการต่อเนื่อง DATA ที่เป็นข้อมูลแถวลำดับปกติ ต้องมีการเพิ่มเนื้อที่ซึ่งเป็นตัวแปรรายการ (List) ที่บรรจุตัวแปร ArrayList จากนั้นคัดลอกข้อมูลไปยังตัวแปรดังกล่าว จึงเรียกใช้ Collections.sort ได้ ขั้นตอนสุดท้ายจึงคัดลอกข้อมูลกลับสู่ DATA อีกครั้ง ดังนั้นขั้นตอนดังกล่าวนี้ สามารถนำมาสร้างเป็นเมธอดสำหรับการจัดเรียงข้อมูลด้วย Collection.sort ดังตัวอย่างต่อไปนี้

ตัวอย่างที่ 10.4 แสดงการจัดเรียงข้อมูล 31, 11, 71, 10, 16, 91, 77, 12, 55, 50 ด้วยแพ็คเกจ `java.util.Collections` และการนำมาสร้างเป็นเมธอดสำหรับจัดเรียงข้อมูล

```
1. package lesson10collections;
2. import java.util.Collections;
3. import java.util.List;
4. import java.util.ArrayList;
5. public class ListSort {
6.     public static int[] listSort(int DATA[], int N){
7.         List<Integer> list = new ArrayList<Integer>();
8.         //คัดลอกข้อมูล DATA ไปยัง List
9.         for(int i=0;i<N;i++){
10.             list.add(DATA[i]);
11.         }
12.         Collections.sort(list); //จัดเรียงข้อมูลใน List
13.         // คัดลอกข้อมูลกลับคืน DATA
14.         for(int j=0;j<list.size();j++){
15.             DATA[j]=list.get(j);
```

```

16. }
17. return DATA;
18. }
19. public static void main(String args[]){
20. int DATA[]={31, 11, 71, 10, 16, 91, 77, 12, 55, 50};
21. System.out.println("ก่อนการจัดเรียง");
22. for(int i=0;i<DATA.length;i++)
23. System.out.print(DATA[i]+" ");
24. listSort(DATA, DATA.length);
25. System.out.println("\nหลังการจัดเรียง");
26. for(int i=0;i<DATA.length;i++)
27. System.out.print(DATA[i]+" ");
28. }
29. }

```

ผลการรันโปรแกรม

ก่อนการจัดเรียง

31 11 71 10 16 91 77 12 55 50

หลังการจัดเรียง

10 11 12 16 31 50 55 71 77 91

ตัวอย่างที่ 10.5 แสดงการจัดเรียงข้อมูล “อรุณ”, “กนก”, “ประยงค์”, “นคร”, “ชนิดา”, “กร”, “จารุวรรณ”, “มะลิ”, “เงิน”, “รุ่งเรือง” ด้วยแพ็คเกจของภาษาจาวาด้วยการนำมาสร้างเป็นเมธอดสำหรับจัดเรียงข้อมูล

```

1. package lesson10collections;
2. import java.util.Collections;
3. import java.util.List;
4. import java.util.ArrayList;

```

```

5. public class ListStringSort {
6.     public static String[] listSort(String DATA[], int N){
7.         List<String> list = new ArrayList<String>();
8.         //คัดลอกข้อมูล DATA ไปยัง List
9.         for(int i=0;i<N;i++){
10.            list.add(DATA[i]);
11.        }
12.        Collections.sort(list); //จัดเรียงข้อมูลใน List
13.        // คัดลอกข้อมูลกลับคืน DATA
14.        for(int j=0;j<list.size();j++){
15.            DATA[j]=list.get(j);
16.        }
17.        return DATA;
18.    }
19.    public static void main(String args[]){
20.        String DATA[]={"อรุณ", "กนก", "ประยงค์", "นคร", "ชนิดา",
a.   "กร", "จรรววรรณ", "มะลิ", "ฉิน", "รุ่งเรือง"};
21.        System.out.println("ก่อนการจัดเรียง");
22.        for(int i=0;i<DATA.length;i++)
23.            System.out.print(DATA[i]+" ");
24.        listSort(DATA, DATA.length);
25.        System.out.println("\nหลังการจัดเรียง");
26.        for(int i=0;i<DATA.length;i++)
27.            System.out.print(DATA[i]+" ");
28.    }
29. }

```

ผลการโปรแกรม

ก่อนการจัดเรียง

อรุณ กนก ประยงค์ นคร ขนิดา กร จารุวรรณ มะลิ เงิน รุ่งเรือง

หลังการจัดเรียง

กนก กร ขนิดา จารุวรรณ นคร ประยงค์ มะลิ รุ่งเรือง อรุณ เงิน

ตัวอย่างที่ 10.6 แสดงการจัดเรียงข้อมูลใน Stack

```
1. package lesson10collections;
2. import java.util.ArrayList;
3. import java.util.Collections;
4. import java.util.List;
5. import java.util.Stack;
6. public class SortStackDemo {
7.     public static void main(String[] args) {
8.         Stack<Integer> stk = new Stack<Integer>();
9.         stk.push(50);
10.        stk.push(20);
11.        stk.push(3);
12.        stk.push(150);
13.        stk.push(120);
14.        stk.push(31);
15.        System.out.println(stk);
16.        sortingStack(stk);
17.        System.out.println(stk);
18.    }
19.    public static Stack sortingStack(Stack source){
20.        Stack<Integer> temStk = new Stack<Integer>();
21.        Object data[] = source.toArray();
22.        List<Integer> list = new ArrayList<Integer>();
23.        for(int i=0; i<data.length;i++) list.add((int)data[i]);
```

```

24. Collections.sort(list);
25. source.clear();
26. for(int i=0;i<list.size();i++) source.push(list.get(i));
27. return source;
28. }
29. }

```

ผลการรันโปรแกรม

[50, 20, 3, 150, 120, 31]

[3, 20, 31, 50, 120, 150]

10.3 การประยุกต์ในการจัดเรียงและการค้นหา

ต่อไปนี้จะแสดงตัวอย่างการประยุกต์ใช้คอลเลกชัน คู่กับสแตดิกคอลเลกชัน เพื่อใช้ในการจัดเรียงและค้นหาข้อมูล ด้วยการแสดงตัวอย่างของข้อมูลที่จัดเก็บในโครงสร้างข้อมูลแบบต่างๆ ที่ได้นำเสนอผ่านมาแล้ว

ตัวอย่างที่ 10.7 แสดงการใช้งาน Collection ร่วมกับ Collections การจัดเรียงข้อมูล

```

1. package lesson10collections;
2. import java.util.ArrayList;
3. import java.util.Collection;
4. import java.util.Collections;
5. import java.util.Comparator;
6. import java.util.LinkedList;
7. import java.util.List;
8. import java.util.PriorityQueue;
9. import java.util.Stack;
10. import java.util.TreeSet;
11. import java.util.Vector;
12. public class CollectionsSort {

```

```

13. public static void main(String args[]){
14. Collection cAL = new ArrayList<Integer>();
15. Collection cTS = new TreeSet<Integer>();
16. Collection cLL = new LinkedList<Integer>();
17. Collection cV = new Vector<Integer>();
18. Collection cST = new Stack<Integer>();
19. Collection cPQ = new PriorityQueue<Integer>();
20. cAL.add(1); cAL.add(2); cAL.add(3); cAL.add(4); cAL.add(5);
21. cTS.add(6); cTS.add(7); cTS.add(8); cTS.add(9); cTS.add(10);
22. cLL.add(11); cLL.add(12); cLL.add(13); cLL.add(14); cLL.add(15);
23. cV.add(16); cV.add(17); cV.add(18); cV.add(19); cV.add(20);
24. cST.add(21); cST.add(22); cST.add(23); cST.add(24); cST.add(25);
25. cPQ.add(26); cPQ.add(27); cPQ.add(28); cPQ.add(29); cPQ.add(30);
26. System.out.println(" cAL:"+cAL);
27. System.out.println(" cTS:"+cTS);
28. System.out.println(" cLL:"+cLL);
29. System.out.println(" cV:"+ cV);
30. System.out.println(" cST:"+ cST);
31. System.out.println(" cPQ:"+cPQ);
32. Collection combine = new LinkedList<Integer>();
33. combine.addAll(cAL); combine.addAll(cTS); combine.addAll(cLL);
    combine.addAll(cV); combine.addAll(cST); combine.addAll(cPQ);
34. System.out.println(combine);
35. List list = new ArrayList<Integer>();
36. list.addAll(combine);
37. Collections.sort(list);
38. System.out.println(list);
39. Collections.sort(list, Comparator.reverseOrder() );
40. System.out.println(list);
41. }

```

42. }

ผลการรันโปรแกรม

cAL:[1, 2, 3, 4, 5]

cTS:[6, 7, 8, 9, 10]

cLL:[11, 12, 13, 14, 15]

cV:[16, 17, 18, 19, 20]

cST:[21, 22, 23, 24, 25]

cPQ:[26, 27, 28, 29, 30]

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30]

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30]

[30, 29, 28, 27, 26, 25, 24, 23, 22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]

ตัวอย่างที่ 10.8 แสดงการค้นหาข้อมูลด้วย Collections

1. package lesson10collections;
2. import java.util.ArrayList;
3. import java.util.Collection;
4. import java.util.Collections;
5. import java.util.Comparator;
6. import java.util.LinkedList;
7. import java.util.List;
8. import java.util.PriorityQueue;

```

9. import java.util.Stack;
10. import java.util.TreeSet;
11. import java.util.Vector;
12. public class CollectionsSearch {
13. public static void main(String args[]){
14. Collection cAL = new ArrayList<Integer>();
15. Collection cTS = new TreeSet<Integer>();
16. Collection cLL = new LinkedList<Integer>();
17. Collection cV = new Vector<Integer>();
18. Collection cST = new Stack<Integer>();
19. Collection cPQ = new PriorityQueue<Integer>();
20. cAL.add(1); cAL.add(2); cAL.add(3); cAL.add(4); cAL.add(5);
21. cTS.add(6); cTS.add(7); cTS.add(8); cTS.add(9); cTS.add(10);
22. cLL.add(11); cLL.add(12); cLL.add(13); cLL.add(14); cLL.add(15);
23. cV.add(16); cV.add(17); cV.add(18); cV.add(19); cV.add(20);
24. cST.add(21); cST.add(22); cST.add(23); cST.add(24); cST.add(25);
25. cPQ.add(26); cPQ.add(27); cPQ.add(28); cPQ.add(29); cPQ.add(30);
26. Collection combine = new LinkedList<Integer>();
27. combine.addAll(cAL); combine.addAll(cTS); combine.addAll(cLL);
    combine.addAll(cV); combine.addAll(cST); combine.addAll(cPQ);
28. System.out.println(combine);
29. List list = new ArrayList<Integer>();
30. list.addAll(combine);
31. //Searching
32. int postion1 = Collections.binarySearch(list, 5);
33. int postion2 = Collections.binarySearch(list, 15);
34. int postion3 = Collections.binarySearch(list, 25);
35. int postion4 = Collections.binarySearch(list, 0);
36. int postion5 = Collections.binarySearch(list, 40);
37. System.out.println("Postion of 5:"+postion1);

```

```
38. System.out.println("Postion of 15:"+postion2);
39. System.out.println("Postion of 25:"+postion3);
40. System.out.println("Postion of 0:"+postion4);
41. System.out.println("Postion of 40:"+postion5);
42. }
43. }
```

ผลการรันโปรแกรม

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30]

Postion of 5:4

Postion of 15:14

Postion of 25:24

Postion of 0:-1

Postion of 40:-31

สรุป

คอลเลกชัน คือ อินเทอร์เฟซที่ให้โครงสร้างชนิดนามธรรมอื่นนำไปอิมพลีเมนต์ใช้งาน อยู่ไลบรารีของภาษา `java.util.Collection` ส่วนคลาสสแตติกคอลเลกชัน `java.util.Collections` มีให้ใช้งานได้โดยไม่ต้องประกาศตัวแปรอ็อบเจกต์ใหม่ คอลเลกชันมีให้ใช้ดำเนินการกับข้อมูลแบบชุดที่ได้นำเสนอไปแล้วในบทที่ผ่านมา การประยุกต์เบื้องต้นแสดงการจัดเรียงและค้นหาข้อมูลในโครงสร้างชนิดนามธรรมเพื่อจัดการข้อมูลชนิดที่เก็บข้อมูลแบบคอลเลกชันให้ได้เหมาะสมมากขึ้น

แบบฝึกหัดท้ายบท

1. กำหนดข้อมูลให้ดังนี้ ,100 ,35 ,40 ,150 ,15 ,4 8,300 จงเขียนโปรแกรมจัดเรียงข้อมูลดังกล่าวจากน้อยไปมากโดยใช้โครงสร้างข้อมูลคอลเลกชัน
2. จากข้อ 1 จงแสดงการเขียนโปรแกรมจัดเรียงข้อมูลโดยใช้สแตตีกคอลเลกชัน
3. จงเขียนโปรแกรมแสดงการค้นหาข้อมูลใน ข้อ 1 และข้อ 2
4. จงอธิบายว่า โครงสร้างชนิดนามธรรม คอลเลกชัน และสแตตีกคอลเลกชันมีความแตกต่างกันอย่างไร
5. จงอธิบายว่า ArrayList สามารถใช้งานแบบ Collection ได้หรือไม่ จงเขียนโปรแกรมเพื่อประกอบการอธิบาย
6. จงเขียนโปรแกรมคัดลอกข้อมูลจากข้อมูลชนิดนามธรรมต่างชนิดกันให้ไปต่อกันอยู่ โครงสร้างข้อมูลชนิด Collection
7. จากข้อ 6 จงเขียนโปรแกรมจัดเรียงข้อมูล โดยใช้ Collections
8. จงอธิบายว่า สามารถนำ Collection และ Collections ไปใช้งานอะไรได้บ้าง จงเขียนโปรแกรมแสดง อย่างน้อย 5 โปรแกรม
9. จงอธิบายว่า Collections แตกต่างจาก Abstract Class อย่างไร
10. จงแสดงแนวคิดของแอปพลิเคชันปัจจุบันที่สามารถนำ Collection Collections ไปใช้งานได้ มาอย่างน้อย 5 แอปพลิเคชัน